

PARKFLOW

Smart Parking & Live Urban Mobility Platform

Research & Technical Feasibility Report

Market benchmark • Architecture • Parking detection • Anti-fraud • Offline operation • ANPR enforcement
• Smart Route-to-Park • MVP plan

PROJECT
ParkFlow

TEAM
90 Minute Legends

REPORT DATE
24 August 2026

Prepared as a decision-oriented research document for product design, prototype implementation, validation, and hackathon judging readiness.

Table of Contents

1. Executive Summary
2. Project Problem & Design Principles
3. Global Market Benchmark
4. What Existing Systems Prove
5. Recommended ParkFlow Architecture
6. Parking Session Start/Stop Strategy
7. Offline Operation & Synchronization
8. Anti-Fraud & Enforcement
9. GPS, Geofencing & Mobile OS Constraints
10. Live Mobility Map & Smart Route-to-Park
11. Parking Availability Prediction
12. Data, AI & Backend Design
13. Security & Privacy
14. MVP Scope & Demo Flow
15. Testing & Validation
16. Business & Sustainability
17. Risks, Weaknesses & Mitigations
18. Competitive Differentiation
19. Judging-Rubric Alignment
20. Implementation Roadmap
21. Final Recommendations
22. References

1. Executive Summary

ParkFlow is best positioned not as a “GPS parking meter,” but as a modular smart-parking and urban-mobility platform. The recommended architecture deliberately separates convenience signals from enforcement: the phone can help detect a likely parking event, the license plate identifies the vehicle, the backend owns the authoritative parking session, and a mobile ANPR enforcement workflow verifies payment without requiring a camera or sensor at every parking space.

Core design decision

Do not make GPS, Bluetooth, QR scanning, or a phone-only heuristic the legal source of truth. Use them to reduce user effort. Use plate-based sessions plus enforcement to address deliberate non-payment.

The research shows that the building blocks are already proven internationally: app-based zone parking (ParkMobile, PayByPhone, RingGo), ANPR-based automatic entry/exit payment in controlled car parks (PayByPhone and RingGo AutoPay), probabilistic parking availability (EasyPark FIND), parking-aware navigation (INRIX Parking Path), and demand-responsive municipal parking management (SFpark). ParkFlow’s opportunity is to combine these ideas into one municipality-oriented workflow and adapt them to environments where dense parking infrastructure is unavailable or too expensive.

- Recommended MVP: manual Start/End with geofence-assisted suggestions, plate registration, backend billing logic, mobile ANPR enforcement, a live mobility map, and Smart Route-to-Park ranking.
- Treat fully automatic GPS-based Start/End as an experimental convenience layer, not as a guaranteed enforcement mechanism.
- Show parking availability as Low / Medium / High probability unless an authoritative occupancy source exists; do not claim exact live space counts.
- Use real or seeded traffic/incidents for the demo, but keep the map API behind an abstraction layer so Google, TomTom, municipal GIS, or another provider can be swapped later.
- Optimize the demo for a complete journey: plan route -> choose parking zone -> start parking -> verify plate -> end session -> update municipal dashboard.

2. Project Problem & Design Principles

ParkFlow addresses two linked problems: inefficient parking payment/enforcement and poor decision support before the driver reaches the parking area. The platform should reduce dependence on physical meters while still providing municipalities with a reliable verification path and useful mobility data.

2.1 Real-world constraints

- Many streets do not have cameras, parking sensors, gates, or dedicated smart-meter infrastructure.
- A user may have weak or no mobile data, may deny location permissions, or may disable Bluetooth.
- GPS can place a phone inside a parking zone even when the registered vehicle is not actually parked there.
- A dishonest driver may simply avoid starting a parking session unless there is independent enforcement.
- Background-location behavior is constrained by Android/iOS power and privacy policies.
- Traffic and incident data availability varies by city and provider.
- Municipal fines and payment settlement are regulated operational processes and should not be faked in an MVP.

2.2 Design principles

- Software-first, infrastructure-light: avoid requiring hardware at every space.
- Plate-centered identity: the license plate is the stable vehicle identifier.

- Phone signals are probabilistic: use GPS/motion/Bluetooth as convenience signals, never as the only proof.
- Backend is authoritative: state transitions, tariffs, session history, and enforcement status belong server-side.
- Progressive deployment: start with zones + app + patrol enforcement; add sensors/cameras only where ROI justifies them.
- Decision-oriented map: parking, traffic, incidents, route cost, walking distance, and predicted availability should support one decision: “Where should I park and how should I get there?”

3. Global Market Benchmark

System	What it proves	Parking start model	Relevant lesson for ParkFlow
ParkMobile	On-demand zone parking at scale; official site states availability in 600+ U.S. cities.	Driver finds/enters zone, selects plate/duration/payment, taps Start.	Digital parking does not need a physical meter at every transaction, but common on-street flows still depend on user activation. [R1]
PayByPhone	App/website/phone payment plus license-plate registration; off-street AutoPay supports ANPR entry/exit.	On-street: manual location/duration. Off-street: ANPR can start/stop automatically.	Automatic charging is robust when an independent entry/exit sensor exists. [R2][R3]
RingGo	Large UK parking network; AutoPay uses ANPR at selected controlled sites.	Manual for normal on-street; ANPR-triggered AutoPay at enabled car parks.	ANPR automation is proven, but infrastructure is required at the monitored access points. [R4] [R5]
EasyPark FIND	Probability-based guidance toward streets with a higher chance of open parking.	Not a parking-start mechanism; it is a search/recommendation feature.	ParkFlow should present probabilistic availability when exact occupancy is unknown. [R6]
INRIX Parking Path	Combines traffic, incidents, off-street parking, and predictive on-street availability for navigation.	Navigation/parking intelligence layer.	Strong precedent for Smart Route-to-Park as a legitimate mobility feature, not feature bloat. [R7]
SFpark / SFMTA	Demand-responsive pricing driven by occupancy data; pilot evaluation reported reduced parking search time.	Meter/sensor-based municipal management.	Parking data can support pricing and congestion policy, but exact occupancy required sensor data in the pilot. [R8][R9]

3.1 Key market conclusion

The market does not invalidate ParkFlow. It clarifies where ParkFlow must be precise: digital parking and plate-based enforcement are proven; fully automatic on-street parking start without independent infrastructure is not something the cited mainstream systems rely on as their enforcement foundation. Therefore ParkFlow should compete on integration, municipality workflow, infrastructure-light deployment, analytics, and route-to-parking intelligence rather than claiming a magic phone-only parking detector.

4. What Existing Systems Prove

Claim	Evidence-backed conclusion
“Mobile parking can replace many meter interactions.”	Yes. ParkMobile, PayByPhone and RingGo all support phone-based parking payments and plate-based vehicle records.
“Automatic parking can be truly hands-free.”	Yes in controlled environments when ANPR cameras observe

Claim	Evidence-backed conclusion
	entry and exit, as shown by PayByPhone AutoPay and RingGo AutoPay.
“Parking availability can be predicted without exact sensors.”	Yes as a probability/likelihood model. EasyPark FIND explicitly presents high/medium/low chance based on data and statistics.
“Parking and traffic routing can be integrated.”	Yes. INRIX Parking Path explicitly combines traffic, incident and parking availability data.
“Parking pricing can influence availability/congestion.”	Yes. SFpark used occupancy-based pricing and reported improvements in parking search outcomes.
“Phone GPS alone can prove that a registered car is parked.”	No. Location describes the phone, not necessarily the vehicle, and OS/background-location limits reduce reliability.

5. Recommended ParkFlow Architecture

The recommended architecture is a layered system. Each layer has a clear responsibility, which makes the design easier to defend technically and easier to prototype.

Layer	Responsibility	Primary components
Driver Experience	Find a zone, view route/traffic, start/end a session, see cost/history.	Mobile app, map UI, local cache, notifications
Parking Detection Convenience	Estimate whether a parking event likely occurred; prompt or auto-suggest.	Geofence, GPS, speed transition, motion/activity, optional vehicle Bluetooth
Authoritative Parking State	Own session start/end, pricing rules, synchronization, idempotency.	Backend API, database, tariff engine, audit log
Vehicle Identity	Bind sessions and enforcement scans to a vehicle.	Normalized license plate + vehicle record
Enforcement	Independently check whether a parked plate has a valid session.	Patrol/mobile ANPR camera, enforcement app, cached checks
Urban Mobility	Rank parking choices and display conditions around them.	Traffic API, incidents, closures, route matrix, parking zones
Analytics / AI	Estimate demand/availability and produce municipality insights.	Historical sessions, time features, route/traffic features, prediction service
Municipality Operations	Configure zones/rules and monitor activity.	Admin dashboard, roles, reports, heatmaps

One-sentence architecture

The phone assists detection, the plate identifies the vehicle, the backend owns the session, and ANPR patrol provides independent enforcement.

5.1 End-to-end flow

1. Driver registers account, vehicle plate and a payment/subscription method.
2. App shows nearby paid zones and Smart Route-to-Park choices.
3. Driver reaches a zone; app suggests the likely zone using location/geofencing.
4. Driver starts parking (MVP baseline) or accepts an auto-detected suggestion.
5. Backend creates an ACTIVE session with authoritative timestamps and tariff snapshot.
6. Enforcement patrol scans the plate and queries/caches session validity.

7. Driver leaves; session is ended manually or by a validated exit heuristic.
8. Backend calculates the charge, closes the session, and updates analytics.

6. Parking Session Start/Stop Strategy

6.1 Why “GPS automatically starts the meter” is unsafe as the core claim

A phone may be inside a paid parking geofence while the registered vehicle is not parked. Examples include being a passenger in another vehicle, walking near the zone, stopping in traffic, waiting at the curb, or having the phone carried away from the car. A phone-only detector cannot produce a cryptographic or physical proof that the registered plate occupies a parking space.

6.2 Recommended three-level decision model

Confidence	Action	Example evidence
High	Offer one-tap Start or, in an experimental mode, auto-start with immediate notification and easy correction.	Inside zone + driving-to-stationary transition + dwell + user leaves vehicle context + optional Bluetooth disconnect
Medium	Ask: “Did you park in Zone A?”	Inside zone + stationary, but vehicle identity evidence is weak
Low	Ignore.	User walks through zone or briefly stops near boundary

6.3 MVP recommendation

- Make Start Parking explicit and fast (one tap after zone suggestion).
- Use geofencing to preselect the zone and reduce user work.
- Use automatic detection as a reminder/prompt first; collect validation data before enabling silent auto-start.
- Never allow turning off Bluetooth/GPS to be interpreted as “parking ended.”
- End Parking should be explicit in the MVP; an exit geofence can suggest/confirm ending.

7. Offline Operation & Synchronization

GPS/GNSS positioning can often continue without mobile data, but map tiles, network-assisted location, routing, server checks and actual payment settlement may require connectivity. ParkFlow can support offline session continuity, but must not describe it as offline banking/payment settlement.

Event	Offline behavior	When connectivity returns
Start requested	Create signed/local pending event with device monotonic time + last known zone metadata.	Server validates, assigns authoritative session ID, checks duplicates and suspicious timestamp drift.
Timer running	Display elapsed time locally.	Server continues from accepted start time.
End requested	Store pending end event.	Server validates sequence and closes once.
Enforcement scan	Use cached recent zone/session metadata where policy permits; queue uncertain checks.	Reconcile queued scans and flag cases that could not be verified in time.
Payment	Show pending charge only.	Capture/settle through payment gateway/server.

7.1 Anti-tamper rules

- Use server time as the authoritative clock whenever online.
- Use monotonic elapsed time locally rather than trusting only the editable wall clock.
- Assign a client-generated event UUID and make sync endpoints idempotent so retries cannot double-create sessions.
- Persist an append-only local event log with hash chaining or signed records for the prototype if feasible.
- Flag impossible sequences: end before start, large clock jumps, repeated offline claims, implausible travel between zones.
- Never silently erase an active session because permissions or connectivity were lost.

8. Anti-Fraud & Enforcement

ParkFlow cannot prevent every unpaid parking event without independent observation. The scalable alternative to a camera per space is mobile enforcement: a patrol vehicle or authorized officer scans plates while moving through paid zones.

Threat	Why it matters	Mitigation
User never starts session	Phone-only system cannot force compliance.	ANPR patrol checks plate against active sessions; apply grace period / review workflow.
Bluetooth disabled / old car	Bluetooth is optional and user-controlled.	Do not use Bluetooth as enforcement proof.
GPS spoofing / permission denial	Location is user-controlled and can be inaccurate.	Use it for UX; enforcement remains plate + patrol + backend.
Short stay between patrol passes	Vehicle may leave before being observed.	Risk remains; tune patrol routes/frequency, use targeted hotspot patrols, optional fixed cameras only where justified.
ANPR misread	Wrong plate could create false flag.	Confidence threshold, image-quality check, normalization, second read/manual review.
Offline patrol	Real-time validation may fail.	Cache zone rules/recent validity where appropriate; queue and mark status as uncertain rather than false-violation.

Important limitation to state to judges

A patrol-based system is not continuous surveillance. It reduces infrastructure cost by accepting a probabilistic enforcement model. If a city requires near-100% capture, more fixed infrastructure is required in high-value zones.

9. GPS, Geofencing & Mobile OS Constraints

Android documentation explicitly limits background location updates to protect battery, and geofence transitions can arrive with latency. Android also requires background-location permission for geofencing on modern versions. This is why the ParkFlow parking detector must tolerate delayed events and should not promise second-level precision from a background app. [R10][R11][R12]

Constraint	Implication for ParkFlow
Background location can be delivered only a few times per hour in some background states.	Do not continuously poll GPS as if it were a foreground navigation app.

Constraint	Implication for ParkFlow
Geofence events can have minute-level latency and can be worse while stationary.	Use dwell windows and user confirmation; do not bill from a geofence timestamp blindly.
Precise location is permission-dependent; approximate location may be much coarser.	Parking-zone selection needs a fallback to manual zone choice or clear map confirmation.
Network-location dependency can affect geofence behavior in poor-connectivity areas.	Cache zone geometry and allow explicit start/stop offline.
Battery cost increases with near-real-time tracking.	Use event-driven geofences/activity transitions, not full-time high-rate GPS.

10. Live Mobility Map & Smart Route-to-Park

The map should not become a mini-Waze. Its scope is parking-oriented decision support: help the driver select the best parking zone and reach it while avoiding poor traffic conditions and incidents.

10.1 Recommended map layers

- Parking zones: boundary, price, hours, restrictions, max stay.
- Traffic: low/medium/heavy road condition or traffic-aware route duration.
- Incidents: accidents, hazards, roadworks, closures; include source and freshness.
- Parking availability: probability category (High / Medium / Low) unless authoritative occupancy exists.
- Walking distance/time from parking zone to final destination.
- Smart recommendation: ranked parking alternatives with an explanation.

10.2 Route-to-Park scoring model

For each candidate parking zone z , compute a normalized score. Lower total cost is better:

$$\text{Score}(z) = w_1 \cdot \text{DriveETA} + w_2 \cdot \text{TrafficPenalty} + w_3 \cdot (1 - \text{AvailabilityProb}) + w_4 \cdot \text{WalkTime} + w_5 \cdot \text{Price} + w_6 \cdot \text{IncidentRisk}$$

Weights can be fixed for the MVP and later personalized. The recommendation UI should explain the top factors instead of showing an unexplained AI score.

10.3 Data/API options

Option	Useful capabilities	ParkFlow use
Google Maps Routes API	Traffic-aware and traffic-aware-optimal routing; route matrices; traffic intervals on polylines.	Compute driver ETA to several parking zones and walking route to destination. [R13][R14]
TomTom Traffic API	Traffic incidents plus real-time flow/speeds/travel times.	Map traffic overlay, incidents and congestion features. [R15]
Municipal / GIS feeds	Official closures, zones, parking regulations, planned works.	Authoritative local data when a municipality partner exists.
ParkFlow reports	Staff/user incident reports with confidence and expiry.	Prototype supplement when official incident feeds are unavailable.

11. Parking Availability Prediction

Without sensors or cameras that directly observe each space, ParkFlow should predict availability rather than claim exact live occupancy. This approach is defensible and consistent with EasyPark FIND, which presents a probability-based color system. [R6]

Feature group	Examples
Time	Hour, weekday/weekend, season, holiday
Parking history	Session starts/ends, average dwell, active sessions, turnover
Zone attributes	Capacity estimate, price, restrictions, land use
Mobility	Nearby traffic level, road closure, incident count
Events	University schedules, market/event periods, local venue events where available
Weather (optional)	Rain/temperature if a reliable source is added later

11.1 MVP model

- Start with a transparent heuristic or Gradient Boosting model using time + historical occupancy proxy.
- Output probability and category: High (>0.65), Medium (0.35–0.65), Low (<0.35) as demo thresholds, then recalibrate from data.
- Evaluate with time-split validation, not random shuffling.
- Display confidence/data freshness; do not pretend sparse demo data is city-scale truth.

12. Data, AI & Backend Design

12.1 Core entities

Entity	Key fields
users	id, role, contact, payment_customer_ref
vehicles	id, user_id, normalized_plate, country/plate_format, status
parking_zones	id, polygon, tariff_id, rules, hours, municipality_id
parking_sessions	id, vehicle_id, zone_id, start_at, end_at, status, fee, source, sync_state
tariffs	id, zone_id, pricing_rule, currency, effective_from/to
enforcement_scans	id, plate_text, confidence, zone_id, captured_at, result, evidence_ref
incidents	id, type, geometry, severity, source, confidence, start_at, expires_at
traffic_snapshots	road/segment, speed, congestion_level, observed_at, provider
parking_predictions	zone_id, horizon, probability, model_version, created_at
audit_log	actor, action, object_type/id, timestamp, metadata

12.2 API endpoints for MVP

- POST /sessions/start, POST /sessions/{id}/end, GET /sessions/active?plate=
- GET /zones/nearby, GET /zones/{id}/rules
- POST /enforcement/scan, GET /enforcement/plate/{plate}/status

- GET /mobility/routes?destination=&candidateZones=
- GET /incidents, POST /incidents/report (authorized or rate-limited)
- GET /predictions/parking?zone=&horizon=
- GET /admin/metrics and /admin/heatmap

12.3 AI components

Component	Recommendation
ANPR	Plate detector + OCR + normalization + confidence threshold; uncertain reads require review.
Parking demand	Time-series safe model with historical session features; probability output.
Traffic prediction	Optional for later phase; real-time provider traffic is enough for MVP.
Incident confidence	Rules/score using source trust, repetition, age, location consistency, and staff verification.
Parking recommendation	Weighted ranking first; ML/ranking model later after enough interaction data.

13. Security & Privacy

ParkFlow handles location, license plates, payments and enforcement evidence. Even in a prototype, privacy-by-design strengthens credibility and prevents the “smart-city surveillance” concern from dominating judging.

- Collect location only when required for the parking/mobility feature; do not build continuous location history by default.
- Separate personally identifiable user data from aggregated analytics.
- Encrypt API traffic; never store raw payment card data in the ParkFlow database.
- Use role-based access: driver, enforcement officer, municipality admin, system admin.
- Keep an audit log for plate lookups and administrative changes.
- Define retention: keep raw ANPR images only as long as required for controlled verification; retain normalized scan metadata longer only if justified.
- Mask plate numbers in broad analytics views.
- Rate-limit public incident reporting and sensitive lookup endpoints.
- Use signed authentication tokens and server-side authorization on every admin/enforcement endpoint.

14. MVP Scope & Demo Flow

MVP priority

A working end-to-end flow scores better than an overextended feature list. The core demo should prove parking session correctness, plate verification, map usefulness, and dashboard synchronization.

14.1 Must work

- Driver account + vehicle plate registration.
- Parking-zone map with rules/prices.
- Start and End Parking with live duration/cost.
- Backend session state machine and pricing.

- ANPR scan -> normalized plate -> session validity result.
- Enforcement interface showing Active / Unpaid / Uncertain.
- Traffic/incidents/closure markers or overlay.
- Smart Route-to-Park ranking between 2–3 candidate zones.
- Municipality dashboard with sessions, revenue demo metrics, scans and demand heatmap.

14.2 Nice-to-have after core is stable

- Auto-detection prompt based on geofence + motion.
- Offline start/end queue and reconciliation.
- Parking availability prediction model.
- Incident confidence scoring.
- Advanced traffic prediction.
- Real payment gateway sandbox.

14.3 90-second demo story

9. Open map and choose destination.
10. Show two parking zones: one closer but congested, another slightly farther with better availability.
11. ParkFlow recommends the better overall option and explains why.
12. Start parking; timer and cost begin.
13. Enforcement camera scans the same plate and returns VALID.
14. Scan a second test plate with no active session and return UNPAID / REVIEW.
15. End parking; final cost appears.
16. Open municipal dashboard to show synchronized session and scan data.

15. Testing & Validation

Test	Success target for controlled MVP
Session state machine	100% correct transitions: inactive -> active -> ended; idempotent retry.
Pricing	Expected fee matches test cases across multiple durations/zones.
ANPR	>=90% correct plate recognition in controlled demo set; uncertain reads visibly flagged.
Enforcement latency	Status returned within a few seconds on demo network.
Offline sync	No duplicate sessions/charges after repeated sync retries.
Map consistency	Incident/closure/parking markers match between driver and admin views.
Route recommendation	Top choice exposes ETA, traffic, predicted availability, walk time and price.
Authorization	Driver cannot access enforcement/admin endpoints.
User validation	5–10 drivers complete setup, start/end, map and route tasks; collect task success + confusion points.

16. Business & Sustainability

16.1 Customers

- Municipalities and local authorities
- Universities and campuses
- Hospitals
- Shopping centers
- Private parking operators
- Large mixed-use developments

16.2 Revenue options

- Municipality SaaS/license fee based on covered zones or active vehicles.
- Small transaction/service fee per paid session where regulation permits.
- Enforcement module subscription per patrol device/vehicle.
- Private parking management contracts.
- Analytics/dashboard premium tier.
- Driver premium features only if they add real value; avoid making basic parking compliance paywalled.

16.3 Municipality value proposition

- Reduce reliance on cash handling and physical meter interaction.
- Digitize plate/session records for faster verification.
- Create demand, turnover and revenue analytics.
- Prioritize enforcement toward hotspots rather than random/manual checking.
- Use parking and traffic data together for zone design and future pricing policy.

17. Risks, Weaknesses & Mitigations

Risk / judge attack	Strong response
“How do you know the car actually parked?”	We do not claim phone GPS is proof. The MVP uses explicit start plus geofence assistance; enforcement is plate-based. Auto-detection remains a confidence feature.
“I can turn off Bluetooth/GPS.”	Bluetooth/GPS improve UX only. A patrol scan still finds a parked plate with no active session.
“What if patrol never sees me?”	That is the infrastructure-versus-coverage trade-off. We reduce cost by accepting patrol-based enforcement; high-risk zones can later add fixed cameras/sensors.
“Where do exact free-space counts come from?”	We do not claim exact counts without occupancy infrastructure; we show availability probability.
“Isn’t this just EasyPark/ParkMobile?”	Those prove digital parking; ParkFlow differentiates through one municipality platform combining parking, patrol verification, route-to-park intelligence, incidents and analytics for infrastructure-light deployment.
“Why not just use Google Maps?”	Google can provide maps/routes/traffic, but ParkFlow adds municipal parking rules, session/payment state, enforcement status, predicted zone availability, and parking-specific ranking.
“What happens offline?”	Session intent can be queued locally; server validates/synchronizes later. Payment settlement and authoritative enforcement remain server-side when connectivity is available.

Risk / judge attack	Strong response
“Background GPS is unreliable.”	Correct; Android restricts background updates. That is why we use geofencing as assistance and do not base legal enforcement on continuous phone GPS.

18. Competitive Differentiation

The strongest positioning is not “nobody has ever done mobile parking.” That claim is false and easy to challenge. The defensible differentiation is the integrated architecture and deployment strategy.

- Unified parking + enforcement + traffic/incidents + parking recommendation + municipality analytics.
- Infrastructure-light on-street deployment: no sensor/camera required at every individual space.
- Mobile enforcement vehicle treated as a roaming smart sensor.
- Plate-centered verification, with phone sensing clearly separated from enforcement truth.
- Parking availability represented honestly as prediction when exact occupancy is unavailable.
- Modular API/provider layer so cities can connect existing GIS, cameras, traffic feeds or meters instead of replacing everything.
- AI used in measurable functions: ANPR, demand prediction and ranking—not as a decorative chatbot.

Positioning statement

ParkFlow is a smart parking and urban-mobility operating layer for cities: it helps drivers choose where to park, manages digital parking sessions, helps enforcement verify plates quickly, and converts parking activity into actionable mobility data.

19. Judging-Rubric Alignment

Rubric area	How ParkFlow should score it
Technical Implementation & GitHub (30)	Working app/backend/ANPR/map integration; clean repository; architecture docs; tests; reproducible setup; real commits and issues.
Demo & Product Quality (25)	Fast end-to-end story; stable session timer; plate scan validation; clear map and parking recommendation; polished states/errors.
Problem Value, Business & Sustainability (20)	Municipality cost/infrastructure trade-off, driver time reduction, enforcement efficiency, credible licensing/transaction model.
Real-World Validation & External Engagement (15)	5–10 driver tests, municipality/parking-operator interview if possible, ANPR field tests, competitor evidence and documented feedback.
Team Execution & Responsiveness (10)	Clear module ownership, fast integration, issue tracking, documented decisions and visible iteration after testing.

19.1 Highest-value work before judging

- Prioritize reliable working flows over advanced prediction.
- Collect real validation evidence and screenshots/videos of tests.
- Document the exact limitation of auto-detection and why the architecture separates it from enforcement.
- Build a judge-ready README with one-command setup or a hosted demo.

- Create 5–8 automated backend tests and a repeatable ANPR test set.
- Prepare answers to the four hard questions: true parking detection, short unpaid stays, availability source, and differentiation from existing apps.

20. Implementation Roadmap

Period	Primary deliverables
20–24 Aug	Research, scope lock, architecture, repository, database/API contract, demo-zone definition.
25 Aug–2 Sep	Mobile core + backend: auth/vehicle/zone/session/pricing; baseline map UI.
3–10 Sep	ANPR pipeline + enforcement app/API + controlled plate test set.
11–17 Sep	Traffic/incidents integration + Smart Route-to-Park + municipality dashboard.
18–23 Sep	Parking availability prediction, offline queue, security hardening, integration tests.
24–27 Sep	User validation, bug fixing, performance, demo script, README, evidence collection.
28 Sep	Feature freeze, final deployment, final demo recording/checklist, submission package.

21. Final Recommendations

17. Keep the MVP parking start explicit; make automatic detection a measured convenience experiment.
18. Use plate-based patrol enforcement as the independent anti-fraud layer; be transparent about its coverage limitations.
19. Do not deploy or promise a camera/sensor per space. Add fixed infrastructure only for controlled car parks or high-value hotspots.
20. Design offline mode as queue-and-sync, not as independent offline payment settlement.
21. Build the map around Smart Route-to-Park: destination -> candidate zones -> traffic/incidents -> availability probability -> walking distance -> ranked recommendation.
22. For the demo, use an API/provider abstraction; Google Routes or TomTom Traffic can power traffic-aware routing/incident data where coverage and budget allow.
23. Show availability as probability unless a trustworthy occupancy source exists.
24. Invest in validation: real drivers, controlled ANPR tests, error cases, and a short municipality/operator interview can materially strengthen credibility.
25. Keep the product story simple: “Plan smarter, park digitally, verify by plate, and give cities better mobility data.”

22. References

[R1] ParkMobile - Zone Parking

Official workflow: park, enter/select zone, choose plate/duration/payment, tap Start; 600+ U.S. cities stated on page.

<https://parkmobile.io/parking/zone-parking>

[R2] PayByPhone - How It Works

Official on-street flow using location number and duration.

<https://www.paybyphone.com/drivers/how-it-works>

[R3] PayByPhone - Off-Street Parking / AutoPay

Official ANPR-based automatic entry/exit and automatic payment for supported car parks.

<https://www.paybyphone.com/drivers/off-street-parking>

[R4] RingGo - AutoPay for Operators

Official description of ANPR cameras triggering automatic session start/end.

<https://ringgo.co.uk/parking-operators/our-services/ringgo-autopay>

[R5] RingGo - Where to Park

Official on-street/car-park coverage and map/location use.

<https://ringgo.co.uk/ringgo-autonomous-site/where-to-park>

[R6] EasyPark - FIND

Official probability-based parking availability and smart route guidance.

<https://www.easypark.com/en-se/help/parking-app/find-what-is-find--25492116456220>

[R7] INRIX Parking Path

Combines traffic, incidents, off-street parking and predictive on-street availability.

https://inrix.com/wp-content/uploads/2018/07/INRIX-Parking-Path_Brochure_7_11_18.pdf

[R8] SFMTA - SFpark Pilot Evaluation

Demand-responsive pricing and occupancy data case study.

https://www.sfmta.com/sites/default/files/reports-and-documents/2018/08/sfpark_pilot_project_evaluation.pdf

[R9] SFMTA - Citywide Demand-Responsive Pricing

Reports pilot outcomes including reduced parking search time.

https://www.sfmta.com/sites/default/files/reports-and-documents/2017/12/12-5-17_item_13_citywide_demand-responsive_parking_pricing_and_transportation_code_amendments.pdf

[R10] Android Developers - Access Location in Background

Background location frequency constraints.

<https://developer.android.com/develop/sensors-and-location/location/background>

[R11] Android Developers - Create and Monitor Geofences

Geofence permissions, latency and connectivity considerations.

<https://developer.android.com/develop/sensors-and-location/location/geofencing>

[R12] Android Developers - Request Location Permissions

Precise/approximate and background-location permission requirements.

<https://developer.android.com/develop/sensors-and-location/location/permissions>

[R13] Google Maps Platform - Routes API

Compute Routes/Route Matrix and traffic-aware routing capabilities.

<https://developers.google.com/maps/documentation/routes>

[R14] Google Maps Platform - Traffic Data Trade-offs

TRAFFIC_UNAWARE / TRAFFIC_AWARE / TRAFFIC_AWARE_OPTIMAL behavior.

https://developers.google.com/maps/documentation/routes/config_trade_offs

[R15] TomTom Developer Portal - Traffic API Introduction

Traffic Incidents and Traffic Flow APIs for real-time road conditions.

<https://developer.tomtom.com/traffic-api/documentation/product-information/introduction>

Research note: URLs and product behavior were checked against official vendor/developer sources in August 2026. Market features and API terms can change; re-verify before production procurement or contractual commitments.